

TestMu AI Talent Assessment

SDET-1 | AI-Native Quality Engineering Challenge

Overview

This isn't a theory test. We want to see how you actually work. What tools you reach for, how you think through a problem, and whether AI is genuinely part of how you build things.

There's no single right answer. We care more about your reasoning than your output.

Log every AI tool you use (Claude, ChatGPT, Cursor, etc.) and what you used it for. This is mandatory. We're not checking if you used AI. We're checking how well you used it.

Scenario

Imagine you've just joined the QA team at TestMu AI. We build a web-based test management platform and the team is moving fast. Your first sprint ticket looks like this:

"The team is spending too much time writing and fixing regression tests. Figure out where AI can help: test generation, validation, anything that reduces manual effort across Login, Dashboard, and API. Show us something working."

Task 1: Setup and Scaffold

- **Repo:** Create a public GitHub repo named `testmu-sdet1-[yourname]`. This is where everything lives.
- **Stack:** Playwright, Selenium, or whatever you know well. Java, Python, or TypeScript.

Your call.

- **Structure:** Set up your folder structure before writing a single test. Clean structure tells us how you think.
 - **First Commit:** Your first commit message should mention which AI tool you used to get started and what it helped with.
-

Task 2: Prompt Engineering for Test Generation

Use an LLM to generate test cases for the three modules below. How you prompt matters just as much as what comes out. A vague prompt that gets lucky won't cut it.

Modules to Cover

- **Login:** Valid login, invalid credentials, forgot password, session expiry, brute-force logout.
- **Dashboard:** Widget loading, data accuracy, filter/sort behavior, responsive layout, permission-based visibility.
- **REST API:** Auth token validation, CRUD operations, error handling (4xx/5xx), rate limiting, schema validation.

What to submit

- `prompts.md` with every prompt you used, exactly as you wrote it. No cleaning up after the fact.
- The generated test cases in Gherkin, JSON, or your framework's format.
- A short note per module (3 to 5 lines) on what didn't work first time and what you changed.

We'll run your prompts ourselves. Consistent, well-structured prompts tell us you know what you're doing. That's exactly what we're looking for.

Task 3: LLM Integration in Test Framework

This is the core of the assignment. We want an LLM wired into your actual test code, not a

chatbot on the side. Pick one of the two options below and build it. The call must be real, not mocked.

Pick ONE

- **Option A, Failure Explainer:** When a test fails, automatically send the page state or API response to an LLM and get back a plain English explanation of what broke plus a suggested fix. Attach this to your test report.
- **Option B, Flaky Test Classifier:** After a test run, feed the execution logs to an LLM and have it sort failures into three buckets: real bug, environment issue, or flaky test. Output this as a structured report.

What to submit

- Working code with a real LLM API call.
- A sample output showing the LLM response inside your test results.
- A comment in your code explaining why you picked this option over the other.

Submission

Share your GitHub repo link once done. Make sure it's public and contains all of the below:

- `prompts.md` with your raw prompts from Task 2, exactly as written
- Generated test cases from Task 2
- LLM integration code and sample output from Task 3
- `README.md` with how to run it, and what you'd build next with more time
- `ai-usage-log.md` with every AI tool used, the task it helped with, and what it produced